

SVSM 2008 Mathematical Modeling

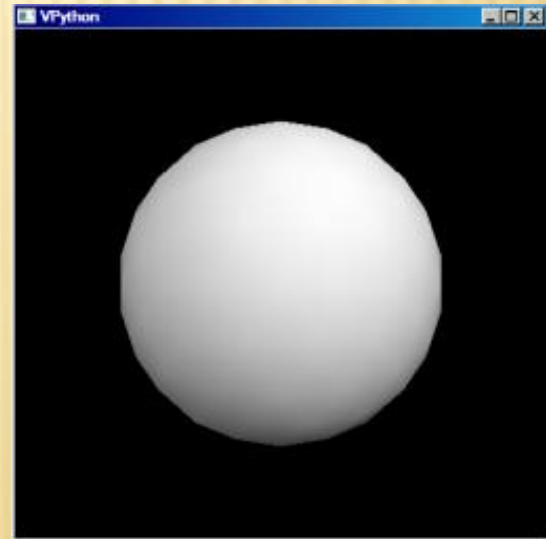
MODELING MOTION IN VPYTHON

VPYTHON = VISUAL PYTHON

- ✖ Easy to learn
- ✖ 3D Interactive modeling
- ✖ Python – background language
- ✖ Main site <http://vpython.org/>
- ✖ Download
http://vpython.org/win_download25.html
- ✖ Python Site <http://www.python.org/>

GETTING STARTED

- ✗ Install Python and then VPython
- ✗ Run IDLE
- ✗ First line
 - + `from visual import *`
- ✗ Next line
 - + `sphere()`
- ✗ Save – **CTRL-S**
 - + Save as `test.py`
- ✗ Run – **F5**



CHANGE SPHERE ATTRIBUTES

- ✖ Color
 - + `sphere(color=color.red)`
- ✖ Radius
 - ✖ `sphere(radius=0.5,color=color.red)`
- ✖ Name
 - + `ball = sphere(radius=0.5,color=color.red)`
- ✖ Position
 - + `ball = sphere(pos=(0,2,0),radius=0.5,color=color.red)`
- ✖ Change position
 - + `ball.pos = (1,2,3)`

NAVIGATION

✖ Zoom

- + Hold middle button and move mouse
- + Or, hold both left and right buttons

✖ Rotate

- + Hold right button

SIMPLE SCENE

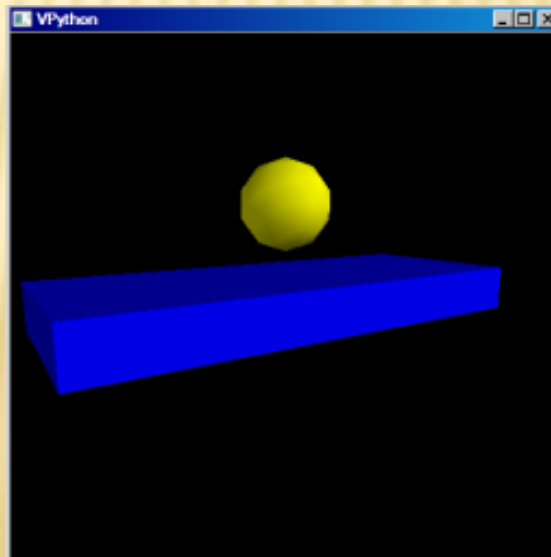
```
from visual import *
```

```
#Create a sphere
```

```
ball = sphere(pos=(0,2,0),color=color.yellow,radius=1)
```

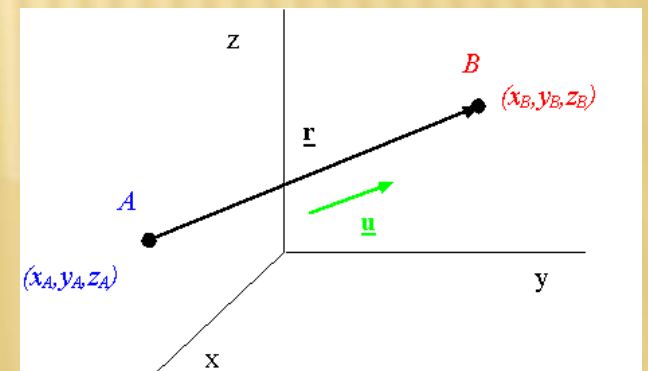
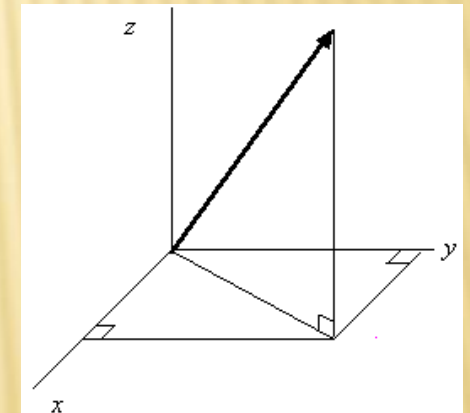
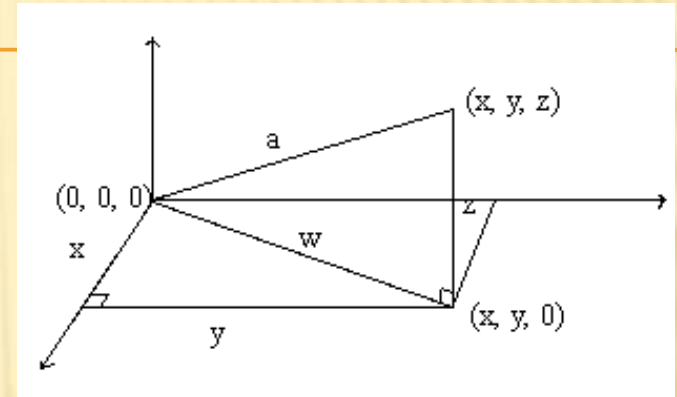
```
#Create a box
```

```
floor = box(length=10, height=2, width=4,color=color.blue)
```



VECTORS

- ✗ Cartesian Coordinates
 - + Locate points with (x,y,z)
- ✗ Vectors
 - + Draw arrow from origin
 - + Vectors have magnitude and direction
 - + $(1,2,3)$
- ✗ General Vector
 - + Draw from one point to another
- ✗ Examples
 - + Position, velocity, force



VECTORS IN VPYTHON

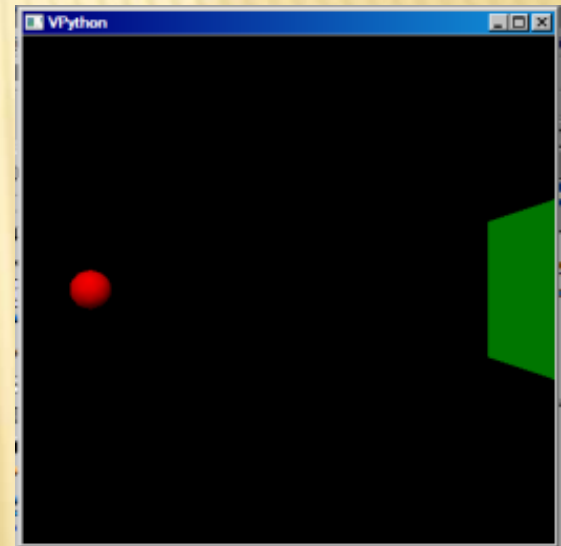
- ✗ Draw arrow from origin to center of ball
 - + `arrow(pos=(0,0,0), axis=ball.pos)`
 - + Begins at pos
 - + Ends at axis
- ✗ Arbitrary arrow
 - + `arrow(pos=(1,2,3), axis=(0,2,-1))`
- ✗ Arrow attributes
 - + `arrow(pos=(1,2,3), axis=(0,2,-1), shaftwidth=1, headwidth=2, headlength=3)`

MOVING BALL – CONSTANT VELOCITY

ADDING MOTION

Goal: Make ball move and bounce off wall

- ✗ Create new program
 - + File – New Window
 - + Save as motion.py
- ✗ Create scene



```
74 *motion.py - C:/Python25/Lib/site-packages/visual/examples/motion.py*
File Edit Format Run Options Windows Help
from visual import *
ball = sphere(pos=(-5,0,0), radius=0.5, color=color.red)
wallR = box(pos=(6,0,0), size=(0.2,4,4), color=color.green)|
```

CONSTANT VELOCITY

✗ Velocity

+ Velocity = displacement/elapsed time

+ $v = \Delta x / \Delta t$

✗ Rewrite

+ $\Delta x = \text{new position} - \text{old position}$

+ $\Delta x = v \Delta t$

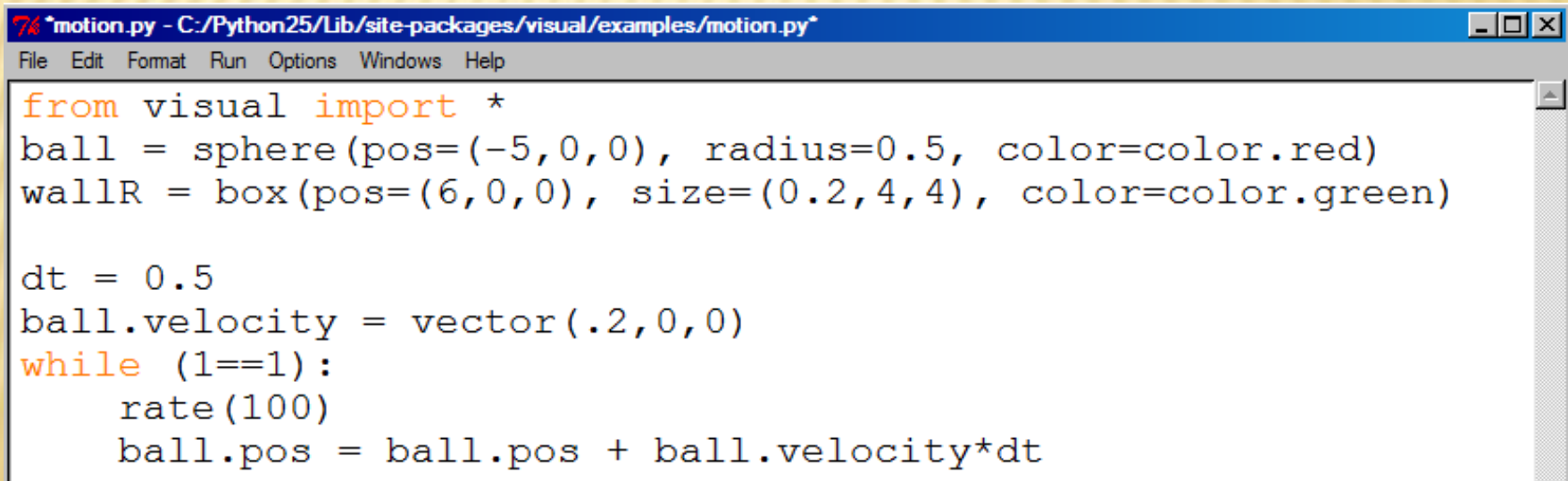
+ new position = old position + $v \Delta t$

✗ In Vpython

+ **$\text{ball.pos} = \text{ball.pos} + \text{ball.velocity} * dt$**

MOVING BALL

- ✗ Need several time steps
 - + Use while loop
 - ✗ Keep executing commands over and over
- ✗ Additional Information
 - + Specify dt and velocity
 - + Need rate – while loop executed 100 times/sec



```
7% motion.py - C:/Python25/Lib/site-packages/visual/examples/motion.py*
File Edit Format Run Options Windows Help

from visual import *
ball = sphere(pos=(-5,0,0), radius=0.5, color=color.red)
wallR = box(pos=(6,0,0), size=(0.2,4,4), color=color.green)

dt = 0.5
ball.velocity = vector(.2,0,0)
while (1==1):
    rate(100)
    ball.pos = ball.pos + ball.velocity*dt
```

MAKING BALL BOUNCE

✗ Logical Tests

- + If the ball moves to far to right, reverse its direction.

✗ If statement

- + **If `ball.x > wallR.x`:**

- + Then what?

✗ Reverse direction

- + **`ball.velocity = -ball.velocity`**

```
while (1==1):  
    rate(100)  
    ball.pos = ball.pos + ball.velocity*dt  
    if ball.x > wallR.x:  
        ball.velocity.x = -ball.velocity.x
```


MODIFY PROGRAM

✖ Assignment 1

- + Add **wallL** on left at (-6,00)
- + Add test to have ball bounce off this wall too.

✖ Assignment 2

- + Change ball velocity to move ball at an angle with no z-component

✖ Assignment 3

- + Add a ceiling and a floor that touch vertical walls
- + Add back wall
- + Add invisible front wall (i.e., only use if statement)
- + Run program with ball bouncing off all walls.

UNIFORM MOTION – ADDING ACCELERATION

FREE FALL

Goal: Drop a ball that bounces on the floor.

- ✗ Free fall

- + All objects near the surface of the Earth fall with an acceleration -9.8 m/s^2 , or -32 ft/s^2 (or $a = -g$).

- ✗ Acceleration

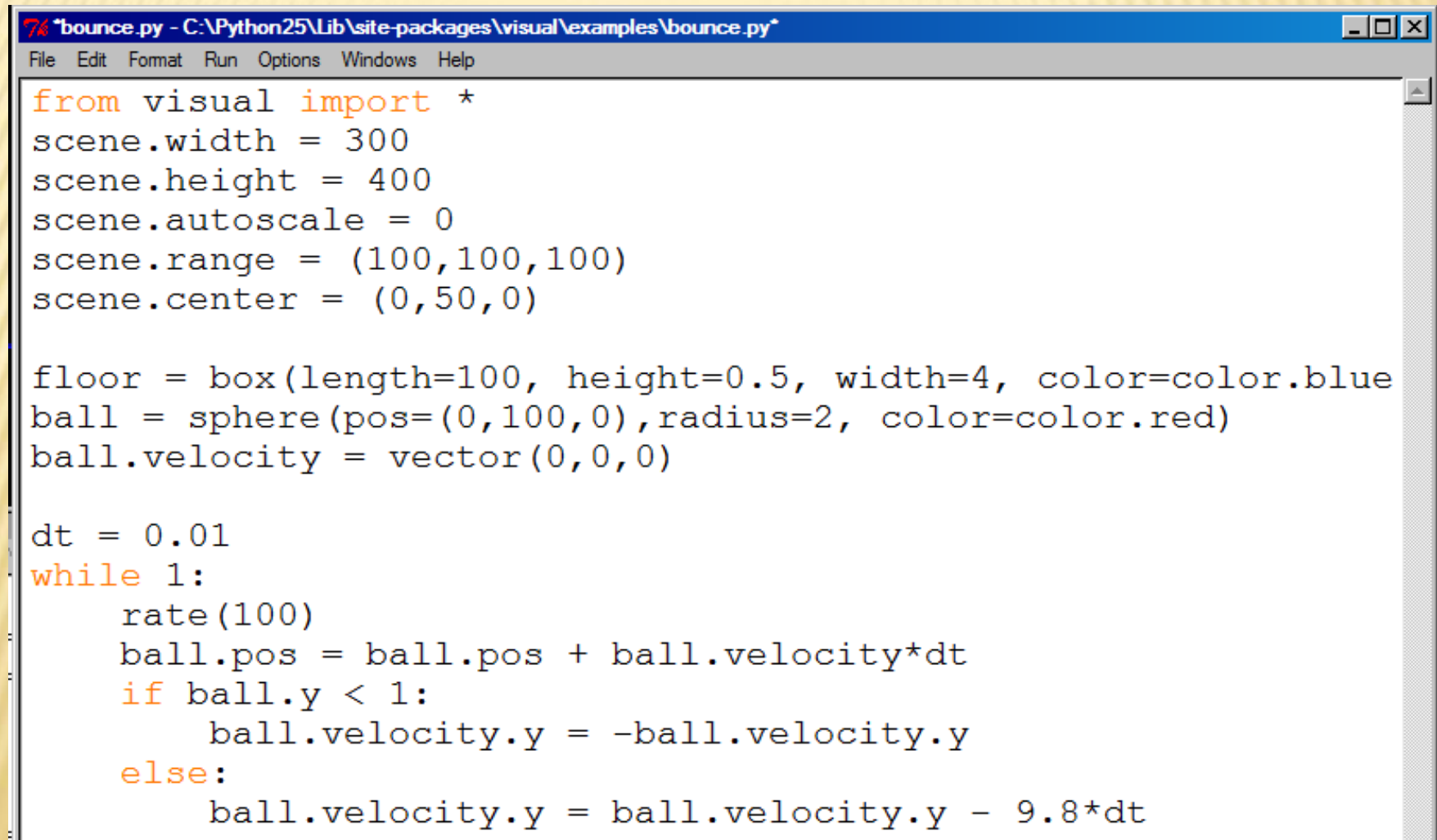
- + $a = \Delta v / \Delta t$

- ✗ Rewrite for VPython

- + new velocity = old velocity + $a\Delta t$

- + **`ball.velocity = ball.velocity + ball.acceleration*dt`**

BOUNCING BALL

A screenshot of a Python script editor window titled "bounce.py - C:\Python25\Lib\site-packages\visual\examples\bounce.py". The window has a menu bar with "File", "Edit", "Format", "Run", "Options", "Windows", and "Help". The script content is as follows:

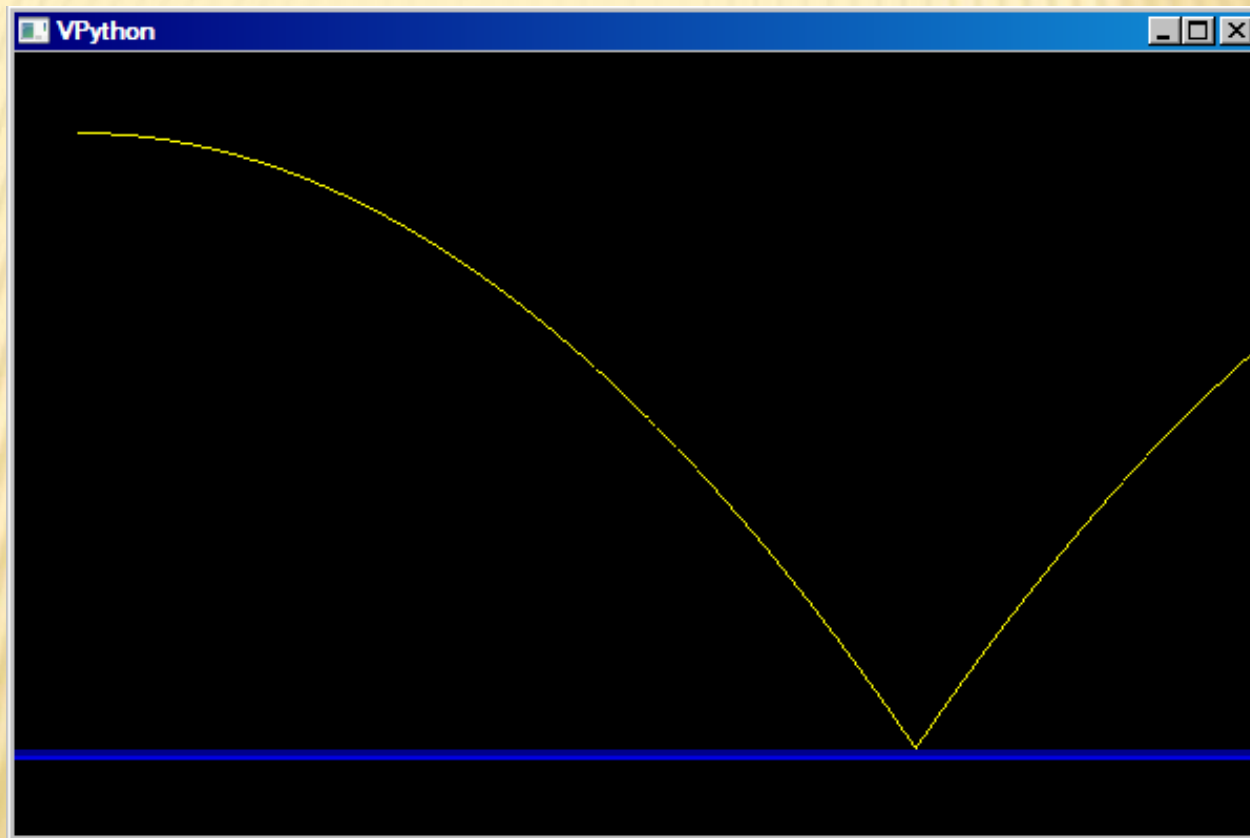
```
from visual import *
scene.width = 300
scene.height = 400
scene.autoscale = 0
scene.range = (100,100,100)
scene.center = (0,50,0)

floor = box(length=100, height=0.5, width=4, color=color.blue)
ball = sphere(pos=(0,100,0), radius=2, color=color.red)
ball.velocity = vector(0,0,0)

dt = 0.01
while 1:
    rate(100)
    ball.pos = ball.pos + ball.velocity*dt
    if ball.y < 1:
        ball.velocity.y = -ball.velocity.y
    else:
        ball.velocity.y = ball.velocity.y - 9.8*dt
```

PROJECTILE MOTION

Free fall with constant horizontal velocity.



PROGRAM FOR PROJECTILE MOTION

7 bounce.py - C:\Python25\Lib\site-packages\visual\examples\bounce.py

File Edit Format Run Options Windows Help

```
from visual import *
scene.width = 600
scene.height = 400
scene.autoscale = 0
scene.range = (100,100,100)
scene.center = (0,50,0)

floor = box(length=300, height=0.5, width=4, color=color.blue)
ball = sphere(pos=(-90,100,0),radius=2, color=color.red)
ball.velocity = vector(3|0,0,0)
ball.trail = curve(color=color.yellow)

dt = 0.01
while 1:
    rate(100)
    ball.pos = ball.pos + ball.velocity*dt
    if ball.y < 1:
        ball.velocity.y = -ball.velocity.y
    else:
        ball.velocity.y = ball.velocity.y - 9.8*dt
    ball.trail.append(pos=ball.pos)
```

ERROR IN CODE?

```
floor = box(length=300, height=0.5, width=4, color=color.blue)
ball = sphere(pos=(-90,100,0),radius=2, color=color.red)
ball.velocity = vector(30,0,0)
ball.trail = curve(color=color.yellow)
```

```
ball2 = sphere(pos=(-90,100,0),radius=2, color=color.red)
ball2.velocity = vector(30,0,0)
ball2.trail = curve(color=color.green)
```

```
ball3 = sphere(pos=(-90,100,0),radius=2, color=color.red)
v = vector(30,0,0)
ball3.trail = curve(color=color.blue)
a = vector(0,-9.8,0)
```

```
dt = 0.01
while 1:
    rate(100)
    ball.pos = ball.pos + ball.velocity*dt
    if ball.y < 1:
        ball.velocity.y = -ball.velocity.y
    else:
        ball.velocity.y = ball.velocity.y - 9.8*dt
    ball.trail.append(pos=ball.pos)

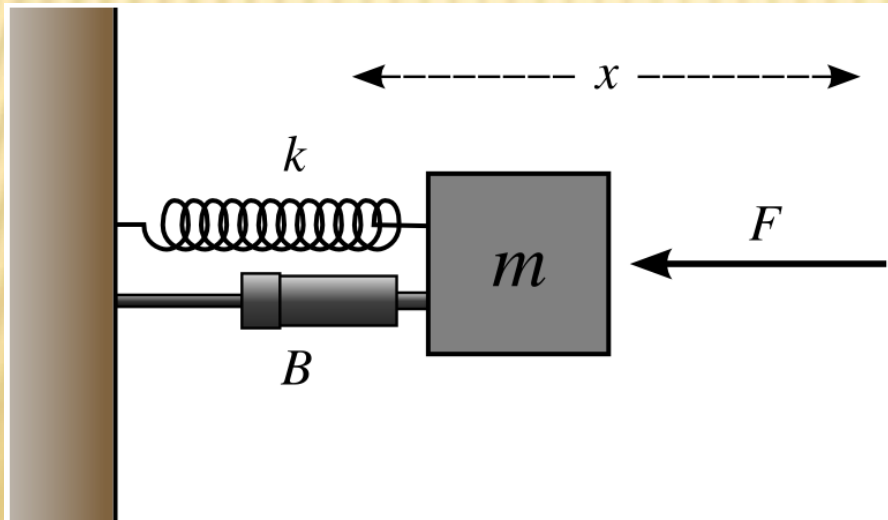
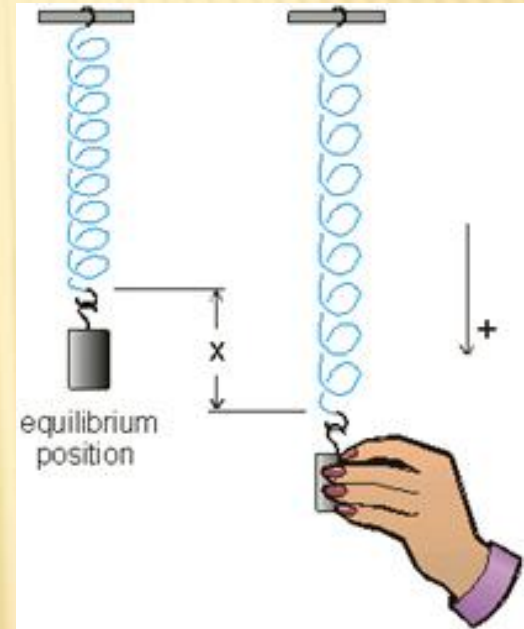
ball.pos = ball.pos + ball.velocity*dt
vel0=ball2.velocity
if ball2.y < 1:
    ball2.velocity.y = -ball2.velocity.y
else:
    ball2.velocity.y = ball2.velocity.y - 9.8*dt
ball2.pos = ball2.pos + (vel0+ball2.velocity)*dt/2
ball2.trail.append(pos=ball2.pos)

if ball3.y < 1:
    v.y= -v.y
else:
    v += a * dt
ball3.pos += v* dt + .5 * a * dt**2
ball3.trail.append(pos=ball3.pos)
```

MASS – SPRING SYSTEM

MASS-SPRING SYSTEM

- ✗ Hooke's Law for Springs
 - + $F = -kx$
- ✗ Newton's 2nd Law of Motion
 - + $F = ma$
 - + Or, $a = F/m$



CREATE THE SCENE

from visual import *

scene.range = (1,1,1)

scene.center = (0,.1,0)

scene.width = 800

scene.height = 275

scene.background = (1,1,1)

x0 = vector(.65,0,0)

Surface = box(size=(2,.02,.5),pos=(0,-.1,0))

wall = box(size=(.04,.5,.3),pos=(-.77,.15,0))

spring = helix(pos=(-.75,0,0),axis=x0,
radius=.08,coils=6,thickness=.01,color=color.blue)

block = box(pos=(0,0,0),size=(.2,.2,.2),color=color.red)

INITIALIZE

`k = 10`

`m = 10`

`b=2`

`v = vector(0,0,0)`

`a = vector(0,0,0)`

`F = vector(0,0,0)`

`block.pos=(0.25,0,0) # Set Initial position of block and connect spring`

`x = block.pos`

`spring.axis = x0 + x`

ADD MOTION

finished = False

dt = .01

while not finished:

rate(100)

$F = -k \cdot x - b \cdot v$

$a = F/m$

$v += a * dt$

$x += v + .5 * a * dt^2$

block.pos = x

spring.axis = x0 + x

CHANGE PROGRAM

- ✗ What happens when the mass is changed?
- ✗ What happens when k is changed?
- ✗ Change the initial block position
 - + i.e., `block.pos=(0.25,0,0)`
 - + Do you need to change other settings?

MAKE UP YOUR OWN MOVING SYSTEM

FREE FOR ALL #1

Play with Vpython and create your own moving objects:

- ✖ Use walls, surfaces, and other objects
- ✖ Other built-in objects

<http://vpython.org/webdoc/visual/index.html>

Cylinder, arrow, cone, pyramid, sphere, ring,
box, ellipsoid, helix

FREE FOR ALL #2

Seach for Vpython Code and see what it does at
<http://people.uncw.edu/hermanr/SVSM.htm>